RESTFUL APPLICATION PROGRAMMING INTERFACE (API) SPECIFICATION

CampusConnect: University Club, Event & Activity Management Platform

Attribute	Specification Detail
Document Identifier	CC-API-2026-V1.0 (IEEE Std 830-1998 / RFC 7807 Standard)
Engineering Version	Version 1.0 (API Contract Baseline)
Software Engineering Team	Md. Shakhawat Hossain (Student ID: 0432320005101009) | Fahim Hossain (Student ID: 0432320005101007) | Department of CSE
Course Advisor & Supervisor	Al Imtiaz | Department of Computer Science and Engineering
Reviewing Body	Faculty Academic Review Board | Division of Computing and Systems
Release Date & Status	October 2026 | Approved for Engineering Implementation

1. Architectural Principles and Protocols

1.1 Overview and Purpose

This document provides the definitive RESTful Application Programming Interface (API) specification for CampusConnect, a centralized university extracurricular and club management platform. It defines all HTTP contracts, authentication schemas, payload definitions, query parameters, authorization tiers, and error envelopes across the 30 server endpoints implemented in the Express.js / TypeScript backend.

1.2 Transport and Data Formatting Standards

- **1. Routing Prefix:** Base URI Prefix: All API endpoints are mounted under the /api/v1 root path.
- **2. Media Type:** Encoding and Payload Format: All request and response bodies utilize application/json with UTF-8 character encoding.
- **3. Authentication:** Stateless Authentication: Stateless JSON Web Tokens (JWT) passed via HTTP Authorization: Bearer <token> headers.
- **4. Refresh Tokens:** Persistent Session Refresh: HTTP-only, Secure, SameSite=Strict cookies protect refresh tokens against Cross-Site Scripting (XSS).
- **5. Error Envelopes:** Standardized Error Envelopes: All client and server errors adhere to RFC 7807 Problem Details formatting.
- **6. Punctuation:** Punctuation Standard: Strictly utilizes ASCII hyphens (-), colons (:), and pipes (|). Zero em-dashes or en-dashes are permitted.

1.3 HTTP Status Codes and Error Model

HTTP Status	Category	Description and Trigger Condition
200 OK	Success	Standard successful read, update, or cancellation operation.
201 Created	Success	New entity created (user registration, event draft, club charter).
400 Bad Request	Client Error	Malformed JSON, expired QR tokens, or deadline passed.
401 Unauthorized	Client Error	Missing, invalid, or expired JWT access token.
403 Forbidden	Client Error	Authenticated user lacks role privileges for the requested action.
404 Not Found	Client Error	Target resource (club, event, registration, certificate) does not exist.
409 Conflict	Client Error	Duplicate registration, membership conflict, or attendance double-scan.
422 Unprocessable	Client Error	Zod schema validation failure on request payload.
429 Too Many Req	Client Error	Rate limit exceeded (>5 failed login attempts in 15 minutes).
500 Server Error	Server Error	Unhandled database or internal server failure.

1.4 RFC 7807 Error Envelope Specification

All error responses return a uniform JSON schema providing actionable error details for client error handlers:

```
{
  "error": "Conflict: You already hold a confirmed registration for this
event",
  "status": 409,
  "timestamp": "2026-10-30T12:00:00.000Z",
  "path": "/api/v1/events/e2b3c4d5-6a7b-8c9d-0e1f-2a3b4c5d6e7f/register"
}
```

2. Module 1: Authentication and User Management

Method & Path	Access	Request Schema	Response Schema & Status
POST /api/v1/auth/register	Public	{ email, password, full_name, student_id_number, department, batch_year, phone_number }	201 Created { message, user: { id, email, role }, profile }
POST /api/v1/auth/login	Public	{ email, password }	200 OK { accessToken, user: { id, email, role, profile } } Set-Cookie: refreshToken
POST /api/v1/auth/refresh	Cookie	Cookie: refreshToken	200 OK { accessToken, user: { id, email, role } }
POST /api/v1/auth/logout	Auth	Headers: Authorization: Bearer <token>	200 OK { message: 'Logged out successfully' } Clear-Cookie: refreshToken
GET /api/v1/auth/me	Auth	Headers: Authorization: Bearer <token>	200 OK { user: { id, email, role, profile } }
GET /api/v1/auth/users	Admin	Query: role, status, page, limit	200 OK { users: [{ id, email, role, status, profile }], total }
PATCH /api/v1/auth/users/:id/role	Admin	{ role: 'STUDENT' | 'CLUB_EXECUTIVE' | 'FACULTY_ADVISOR' | 'ADMIN' }	200 OK { message: 'Role updated', user: { id, email, role } }

3. Module 2: Clubs and Membership Management

Method & Path	Access	Request Schema	Response Schema & Status
GET /api/v1/clubs	Public	Query: category, search	200 OK { clubs: [{ id, name, slug, category, logo_url, member_count }] }

Method & Path	Access	Request Schema	Response Schema & Status
GET /api/v1/clubs/:idOrSlug	Public	Param: idOrSlug (UUID or slug)	200 OK { club: { id, name, description, category, advisor, executives, events } }
POST /api/v1/clubs	Admin	{ name, slug, description, category, logo_url, faculty_advisor_id }	201 Created { message: 'Club created', club: { id, name, slug } }
POST /api/v1/clubs/:clubId/apply	Student	{ reason?: string }	201 Created { message: 'Application submitted', membershipId, status: 'PENDING' }
GET /api/v1/clubs/:clubId/members	Auth	Query: status ('PENDING' | 'APPROVED')	200 OK { members: [{ id, user_id, full_name, student_id_number, club_role, status }] }
PATCH /api/v1/clubs/:clubId/members/:membershipId	Club Exec	{ status: 'APPROVED' | 'REJECTED', club_role?: 'MEMBER' | 'EXECUTIVE' }	200 OK { message: 'Membership status updated', membership: { id, status, club_role } }
GET /api/v1/clubs/:clubId/analytics	Club Exec	Param: clubId	200 OK { totalMembers, activeEvents, totalRegistrations, averageAttendanceRate }

4. Module 3: Event Governance and Lifecycle

Method & Path	Access	Request Schema	Response Schema & Status
GET /api/v1/events	Public	Query: category, clubId, status, search	200 OK { events: [{ id, title, club_name, venue, start_time, capacity, registered_count }] }
GET /api/v1/events/review/pending	Advisor / Admin	Headers: Authorization: Bearer <token>	200 OK { pendingEvents: [{ id, title, club_name, capacity, start_time, created_by }] }

Method & Path	Access	Request Schema	Response Schema & Status
GET /api/v1/events/:id	Public	Param: id (Event UUID)	200 OK { event: { id, title, description, venue, capacity, registered_count, is_registered } }
POST /api/v1/events	Club Exec	{ club_id, title, description, venue, start_time, end_time, capacity, registration_deadline }	201 Created { message: 'Event drafted', event: { id, title, status: 'DRAFT' } }
POST /api/v1/events/:id/submit	Club Exec	Param: id (Event UUID)	200 OK { message: 'Event submitted for advisor review', status: 'SUBMITTED' }
POST /api/v1/events/:id/review	Advisor / Admin	{ status: 'APPROVED' | 'REJECTED', advisor_remarks: string }	200 OK { message: 'Review recorded', status: 'PUBLISHED' | 'REJECTED' }
POST /api/v1/events/:id/cancel	Club Exec / Admin	{ reason: string }	200 OK { message: 'Event cancelled', status: 'CANCELLED' }

5. Module 4: Registrations and Concurrency-Safe Waitlist

Method & Path	Access	Request Schema	Response Schema & Status
POST /api/v1/events/:eventId/register	Student	Param: eventId (Concurrency Safe)	201 Created (Confirmed Slot) { status: 'CONFIRMED', ticketCode, message } OR 200 OK (Waitlisted) { status: 'WAITLISTED', waitlistPosition: 3 }
POST /api/v1/registrations/:registrationId/cancel	Student / Admin	Param: registrationId	200 OK { message: 'Registration cancelled', promotedStudentUserId }

Method & Path	Access	Request Schema	Response Schema & Status
GET /api/v1/registrations/me	Student	Headers: Authorization: Bearer <token>	200 OK { registrations: [{ id, event_title, venue, status, ticket_code, waitlist_position }] }
GET /api/v1/events/:eventId/roster	Club Exec / Advisor	Param: eventId	200 OK { confirmed: [{ user_id, full_name, ticket_code }], waitlisted: [...] }

6. Module 5: Dynamic Anti-Proxy Attendance Tracking

Method & Path	Access	Request Schema	Response Schema & Status
GET /api/v1/events/:eventId/attendance/live-qr	Club Exec	Param: eventId	200 OK { dynamicToken: 'HMAC_SHA256', expiresInSeconds: 20, timeStep: 172900000 }
POST /api/v1/events/:eventId/attendance/scan	Student	{ dynamicToken: string }	200 OK { message: 'Check-in confirmed', checkInTime: '2026-10-30T12:00:00Z', method: 'DYNAMIC_QR' }
POST /api/v1/events/:eventId/attendance/manual	Club Exec	{ student_id_number: string }	200 OK { message: 'Manual check-in recorded', method: 'ORGANIZER_MANUAL' }
GET /api/v1/events/:eventId/attendance	Club Exec / Advisor	Param: eventId	200 OK { totalCheckedIn: 48, records: [{ user_id, student_name, check_in_time, method }] }

7. Module 6: Certificates and Public Verification

Method & Path	Access	Request Schema	Response Schema & Status
POST /api/v1/events/:eventId/certificates/issue	Club Exec	Param: eventId	200 OK { message: 'Certificates generated', issuedCount: 45, failedCount: 0 }
GET /api/v1/certificates/me	Student	Headers: Authorization: Bearer <token>	200 OK { certificates: [{ id, serial_number, event_title, issue_date, pdf_url }] }
GET /api/v1/certificates/download/:serialNumber	Public	Param: serialNumber	200 OK Content-Type: application/pdf (High-resolution official credential stream)
GET /api/v1/certificates/verify/:serialNumber	Public (No Auth)	Param: serialNumber	200 OK { status: 'VALID', recipientName, studentId, eventTitle, clubName, issueDate, sha256 }

8. Module 7: Announcements, Notifications and Feedback

Method & Path	Access	Request Schema	Response Schema & Status
GET /api/v1/clubs/:clubId/announcements	Public	Param: clubId	200 OK { announcements: [{ id, title, content, author_name, created_at }] }
POST /api/v1/clubs/:clubId/announcements	Club Exec	{ title: string, content: string }	201 Created { message: 'Announcement published', id, created_at }
GET /api/v1/notifications	Auth	Headers: Authorization: Bearer <token>	200 OK { notifications: [{ id, title, message, link_url, is_read, created_at }] }
PATCH /api/v1/notifications/:id/read	Auth	Param: id (Notification UUID)	200 OK { message: 'Marked as read', is_read: true }
POST /api/v1/events/:eventId/feedback	Student (Attended)	{ rating: 1..5, comments?: string, is_anonymous?: boolean }	201 Created { message: 'Feedback submitted successfully' }

Method & Path	Access	Request Schema	Response Schema & Status
GET /api/v1/events/:eventId/feedback	Club Exec / Advisor	Param: eventId	200 OK { averageRating: 4.8, totalSubmissions: 42, feedbackList: [...] }

9. Module 8: Institutional Analytics and Operational Health

Method & Path	Access	Request Schema	Response Schema & Status
GET /api/v1/analytics/admin	Sys Admin	Headers: Authorization: Bearer <token>	200 OK { totalUsers, activeClubs, totalEvents, totalRegistrations, certificatesIssued }
GET /api/v1/health	Public	None	200 OK { status: 'HEALTHY', timestamp: '2026-10-30T12:00:00Z', platform: 'CampusConnect v2.0' }